

Practical Assignment

Yair Schiff

yzs2

*Released: April 13, 2026**Due: May 5, 2026 (extended)*

Responses formatted in **blue** text.

In this assignment, you are invited to implement and test first-order algorithms for solving the following unconstrained minimization problem:

$$\min_{x \in \mathbb{R}^n} \left[f(x) = \frac{1}{m} \sum_{i=1}^m l(\langle a_i, x \rangle - b_i) + \psi(x) \right], \quad (1)$$

which often appears in applications with machine learning and statistics. In problem (1),

$$\mathcal{D} = \{a_1, \dots, a_m \in \mathbb{R}^n, b_1, \dots, b_m \in \mathbb{R}\} \quad (2)$$

is the data that might come from different sources,

$$l : \mathbb{R} \rightarrow \mathbb{R}$$

is a convex loss function, which is part of the model, and is a possible regularizer.

The full solution to this assignment consists of two components:

- A source code in Python¹ of all parts of your implementation (data generator, optimization algorithms, and the code that you used for running all the experiments and producing the plots), *well organized* and *readable*.
- A report in PDF, clearly explaining the experimental setup, generated plots, and discussion of the results. You should check if your experimental results match the theory, or provide a reasonable explanation if it is not the case (e.g., which assumptions were not satisfied, so an algorithm does not achieve a predictable behavior).

Please organize all your files and report into a single ZIP file and submit via the gradescope.

The assignment consists of the following key parts:

1 Data Generation

Implement a function that generates a random instance of data (2), with the following signature²:

¹Please contact me if you strongly prefer to substitute Python with another programming language in your implementation.

²If you want, you can use additional an parameter with a default values set in all functions.

```
def generate_data(n, m, sigma, seed=6365):
    ...
    return (A, b)
```

where $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ is a generated instance (`numpy arrays`), and $\sigma > 0$ is a parameter of *ill-conditioning* of the data. Namely, we want to have a control over the following quantity:

$$\sigma = \lambda_{\max}(B)/\lambda_{\min}(B), \quad (3)$$

where $B := A^\top A$. So, the way you generate A and b should satisfy (3) up to a reasonable precision, and you should return the same random data for different runs of the function with a fixed `seed`.

My implemented method satisfies (3), up to a reasonable precision, as shown in Table 1.

Table 1: Confirming data generation method respects the desired condition number, up to numerical precision.

n	m	Target σ	Relative Error
100	100	10	$3.553e^{-15}$
100	100	1000	$1.697e^{-13}$
100	100	100000	$1.274e^{-12}$
100	1000	10	$9.237e^{-15}$
100	1000	1000	$1.035e^{-14}$
100	1000	100000	$7.448e^{-12}$

2 Algorithms for Smooth and Strongly Convex Problem

Implement the following methods for solving problem (1) with a smooth loss l and strongly convex regularizer, for some $\mu \geq 0$:

$$\min_{x \in \mathbb{R}^n} \left[f(x) = \frac{1}{m} \sum_{i=1}^m l(\langle a_i, x \rangle - b_i) + \frac{\mu}{2} \|x\|_2^2 \right]. \quad (4)$$

2.1 The gradient method

Please use the following signature:

```
def gradient_method(A, b, loss, mu, x_0, n_iters, L0, adaptive=False):
    ...
    return x_sol, history, status
```

where

- A, b are given data for the problem instance;

- `loss` is a string with the following possible values: `loss = 'quadratic'` which corresponds to linear regression loss: $l(t) = \frac{1}{2}t^2$ and `loss = 'logistic'` which corresponds to the logistic regression loss: $l(t) = \log(1 + e^t)$;
- `mu` is the value of the regularization parameter $\mu \geq 0$;
- `x_0` is the initial point, `n_iters` is the number of iterations to run the method;
- `L0` is an estimate of the Lipschitz constant of the gradient;
- `adaptive=True` or `False` corresponds to whether using an adaptive search for estimating the Lipschitz constant at each iteration of the method, or using the constant value `L0` for all iterations (constant step-size).

The function should return:

- `x_sol` which is a point with the best function value among observed iterates;
- `history` should contain the following information:
 - `history['func']` is the list of function values among all main iterates $\{x_k\}_{k \geq 0}$;
 - `history['grad']` is the list of gradient norms among all main iterates $\{x_k\}_{k \geq 0}$;
 - `history['time']` is the list of time snapshots taken at the start of every iteration;
 - `history['mat_vec']` is the list of total numbers of matrix-vector products Ah and $A^\top h$ used up to each iteration (cumulative statistics);

A good implementation should effectively use matrix and vector operations from `numpy` and `linalg`, avoiding auxiliary loops for computing the function value and the gradient as much as possible. Please also note that all parameters sent to the function *should remain unchanged* (in particular, x_0 should remain unchanged after running the method);

2.2 The fast gradient method

Please use the following signature:

```
def fast_gradient_method(A, b, loss, mu, x_0, n_iters, L0, adaptive=False):
    ...
    return x_sol, history, status
```

with the same meaning of the parameters as for the basic gradient method.

3 Algorithm for Non-Smooth Problems

Implement the projected subgradient method for solving problem (1) with smooth and non-smooth losses and a ball constraint, for some $R > 0$:

$$\min_{x \in \mathbb{R}^n: \|x\|_2 \leq R} \left[f(x) = \frac{1}{m} \sum_{i=1}^m l(\langle a_i, x \rangle - b_i) \right] \quad (5)$$

Please use the following signature:

```
def subgradient_method(A, b, loss, R, x_0, n_iters, gamma, normalized=False):  
    ...  
    return x_sol, history, status
```

where the differences with the smooth case are the following:

- `loss` is a string with possible values: `loss = 'quadratic'`, `loss = 'logistic'`, or `loss = 'l1'` which corresponds to $l(t) = |t|$;
- `R` is a radius of the ball;
- `gamma` is a step-size parameter in the subgradient method;
- `normalized=True` or `False` corresponds to whether normalize the subgradient direction by its Euclidean norm, or not;

The function should return the same result as in the smooth case, but use for `history['grad']` the norms of the subgradients used over the iterations.

4 Experiments

Consider several instances of the randomly generated data for fixed moderate values n and m (around 100 and around 1000) and different values of the condition number $\sigma = 10, 10^3, 10^5$. For each instance of the generated data, perform the following set of experiments:

1. Run both the gradient method and the fast gradient method on quadratic and on logistic loss, for different values of $\mu \geq 0$ starting from $\mu = 0$, with a constant value of the regularization parameter L (chosen according to the theory).

We choose the constant L for `L_0` by deriving a bound for $\|\nabla^2 f(x)\|$ (see Theorem 3.1.4 from the lecture notes). Specifically, we have:

- **Quadratic Loss:** We derive the Hessian: $\nabla^2 f(x) = \frac{1}{m} A^\top A + \mu I_n$. Thus, we can use $L = \|\frac{1}{m} A^\top A + \mu I_n\|_2$, where we can use $\|A\|_2^2$ instead of $\lambda_{\max}(A^\top A)$. Thus we use the following estimate for `L_0`: $\frac{\|A\|_2^2}{m} + \mu$.

- **Logistic Loss:** We use the same logic as above, except that we have an additional $1/4$ factor that comes from the following: in general, the Hessian is of the form: $\nabla^2 f(x) = \frac{1}{m} A^\top \text{diag}(l''(Ax - b))A + \mu I_n$, and here $l''(t) = \text{sigm}(t)\text{sigm}(1 - t) \leq 1/4$, where $\text{sigm}(\cdot)$ represents the sigmoid function. Thus we can estimate $L = \frac{\|A\|_2^2}{4m} + \mu$.

Plot the graphs of the functional residual $f(x_k) - f^*$ and the gradient norm $\|\nabla f(x_k)\|$ (in logarithmic scale), versus the iteration number k , the computational time `history['time']` and the total number of matrix-vector products `history['mat_vec']` (in standard linear scale). For estimating the minimal function value f^* you might use either the best function value found after the launch of all the algorithms, or use the built-in function `scipy.optimize` in Python to estimate it.

To estimate the minimal function value, f^* , we use the best function value found during the descent method. We report results for vanilla and fast gradient methods applied to the quadratic and logistic loss for $n = 100, \sigma \in \{10, 10^3, 10^5\}$, and $\mu \in \{0, 0.001, 0.01\}$. In Figures 2-4, we report the smaller experiment setting of $m = 100$. In Figures 5-7, we report the larger setting of $m = 1000$.

For all smooth gradient experiments we use 500 as the maximum number of iterations.

Try answering the following questions:

What is the rate of convergence for each of the method?

How is the convergence rate affected by the strong convexity parameter and the data condition number σ ?

We focus our analysis on the larger synthetic setting $n = 100, m = 1000$. For general smooth convex objectives, gradient descent has an $O(1/k)$ guarantee and fast gradient has an accelerated $O(1/k^2)$ guarantee. However, several of our tested problems satisfy stronger assumptions.

In particular, for the quadratic objective, the Hessian is $\frac{A^\top A}{m + \mu I}$. Since the generated matrices have full column rank when $m \geq n$, $A^\top A/m$ is positive definite. Therefore the quadratic objective is strongly convex even when $\mu = 0$; adding $\mu > 0$ increases the curvature and improves the condition number. Therefore, the near-straight lines in the (log)residual plots are consistent with the stronger linear/geometric theory. As σ increases, the condition number worsens, so the linear rate becomes much slower and can look nearly flat over the finite iteration budget. This explains why highly ill-conditioned $\mu = 0$ runs look less cleanly geometric, even though the quadratic objective is still strongly convex. For logistic loss, $\mu > 0$ gives strong convexity through regularization; when $\mu = 0$, global strong convexity is not guaranteed.

Now, to compare the two gradient methods: Fast gradient usually reduces the residual in fewer iterations than gradient descent. However, the matrix-vector plots show that the extra computations in fast gradient can reduce or sometimes offset this advantage. Thus, the iteration plots favor fast gradient, while the matrix-vector plots give a more realistic computational comparison.

2. Repeat the same set of experiments but enabling the adaptive search: `adaptive = True`.

We repeat the settings from above. We report $m = 100$ in Figures 8-10 and $m = 1000$ in Figures 11-13.

Does adaptive search help for a faster convergence?

To facilitate this comparison, we present heatmaps that show relative speed-up between adaptive and non-adaptive step sizes, where values > 1 indicate that the adaptive search reached the relative residual error reduction faster than the fixed step size. We present this comparison for both number of iterations (Figure 1, *top*) and matrix vector operations (Figure 1, *bottom*). We focus on the $m = 1000$ setting. Experiments where neither adaptive nor fixed step sizes reach the target relative reduction are marked by “NR”. Experiments where only adaptive reaches the target reduction are marked by “A”.

Our findings show that for many of the experimental settings adaptive search does not lead to faster convergence. This is possibly due to the fact that the Lipschitz constant we choose is already quite useful. Adaptive search does improve iteration count in easier or less-regularized settings where the fixed global Lipschitz step can be conservative. However, the matrix-vector heatmap shows that this iteration-level gain often disappears or reverses once line-search overhead is counted. Thus, adaptive search is not uniformly better; it is most useful when the fixed Lipschitz estimate is overly conservative, but when accounting for the additional computational overhead of adaptive search these gains are mostly erased (except for the $\sigma = 10, \mu = 0$ setting).

3. Run the subgradient method on quadratic, logistic, and l1 losses, using normalized and non-normalized step-sizes, with γ chosen to be a constant, according to theory, for few different values of the radius R and the number of iterations.

To choose γ , we follow the lecture notes. In `normalized = True`, we simply use, $\gamma = R/\sqrt{K}$, for $K = \text{n_iters}$. For unnormalized subgradient steps, we further scale this down by a factor M : $\gamma = R/(M\sqrt{K})$, where M is a bound on the subgradient norm. For logistic and l1 losses, the gradient with respect to the residual $\langle a_i, x \rangle - b_i$ is bounded by 1, hence for subgradient $g(x)$, letting r denote a vector with $r_i = \ell'(\langle a_i, x \rangle - b_i)$, with $|r_i| \leq 1$, we have

$$g(x) = \frac{1}{m} A^\top r \implies \|g(x)\|_2 \leq \frac{1}{m} \|A\|_2 \|r\|_2 \leq \frac{1}{m} \|A\|_2 \sqrt{m} = \frac{\|A\|_2}{\sqrt{m}}.$$

For quadratic loss,

$$g(x) = \frac{1}{m} A^\top (Ax - b) \implies \|g(x)\| \leq \frac{1}{m} \|A\|_2 \|Ax - b\| \leq \frac{1}{m} \|A\|_2 (\|A\|_2 R + \|b\|).$$

We re-run the experiments from above varying $R \in \{1, 10\}$ and $K \in \{500, 2000\}$. In Figures 14-16, we report experiments for $R = 1, n = 100, m = 100$. In Figures 17-19, we report experiments for $R = 10, n = 100, m = 100$. In Figures 20-22, we report experiments for $R = 1, n = 100, m = 1000$. In Figures 23-25, we report experiments for $R = 10, n = 100, m = 1000$.

Try answering the following questions:

What is the rate of convergence of the subgradient method? Compare the obtained plots with the previous ones for the methods on smooth instances.

The projected subgradient method has a generic sublinear guarantee, $O(1/\sqrt{K})$, for the best observed objective residual with an appropriate constant step size. The l1 experiments are most representative of this: the residual curves are noisy, improve slowly, and do not show

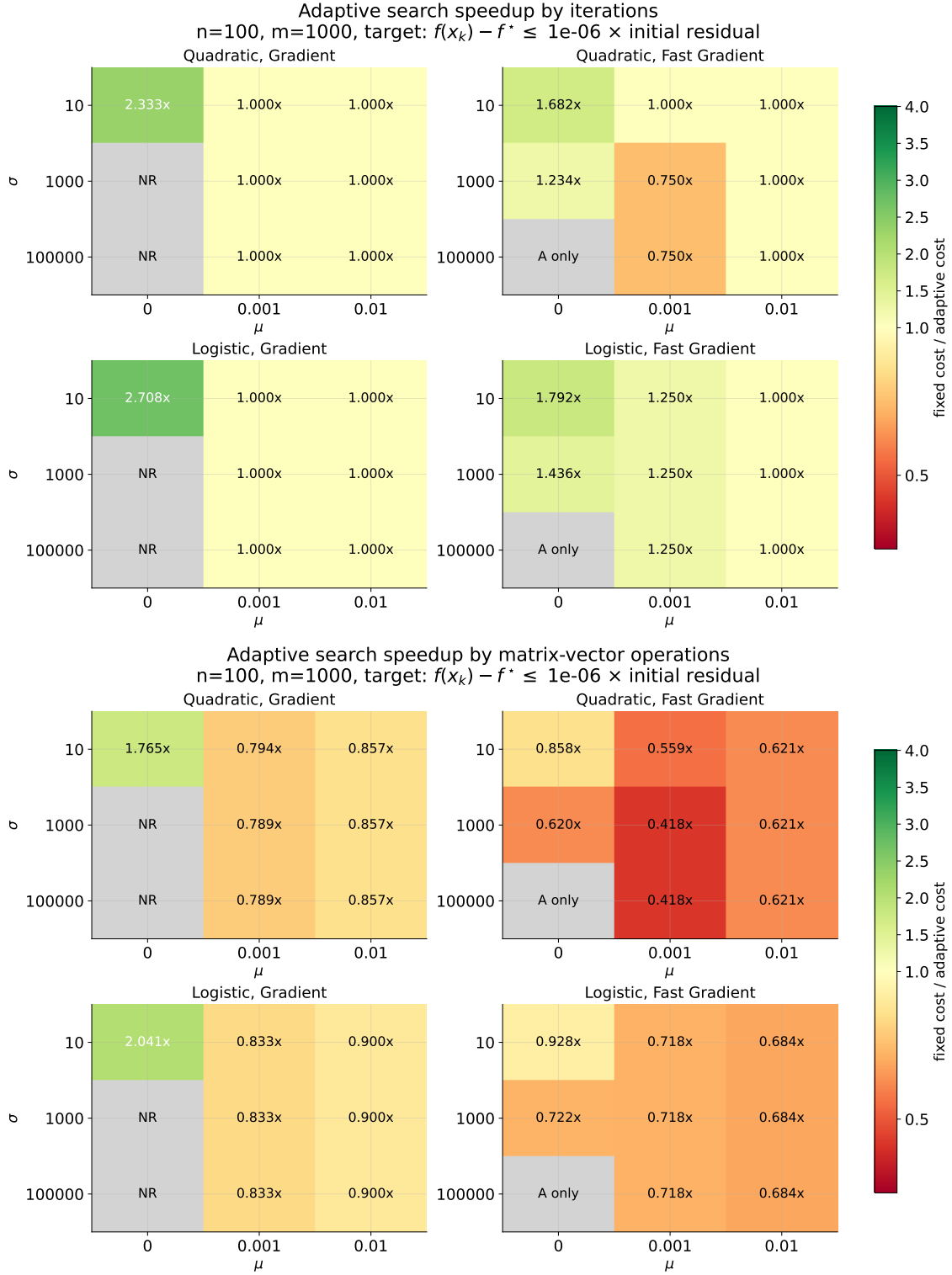


Figure 1: Relative speed-up, in terms of iterations (*top*) and matrix-vector operations (*bottom*), of adaptive search.

stable geometric decay. Additionally, in contrast to the smooth gradient and fast-gradient plots, the subgradient norms are noisy and not generally monotone.

However, the quadratic and logistic losses are smooth, and the generated instances often satisfy stronger curvature conditions on the feasible ball. For those cases, the same projected subgradient code behaves more like a projected gradient method, and the residual curves can display approximately linear-looking phases during portions of the descent method.

Does normalization of subgradients help for a faster convergence?

Normalization generally helps. The normalized runs often reached small objective residuals in fewer iterations than the unnormalized runs.

5 Bonus

Skipped.

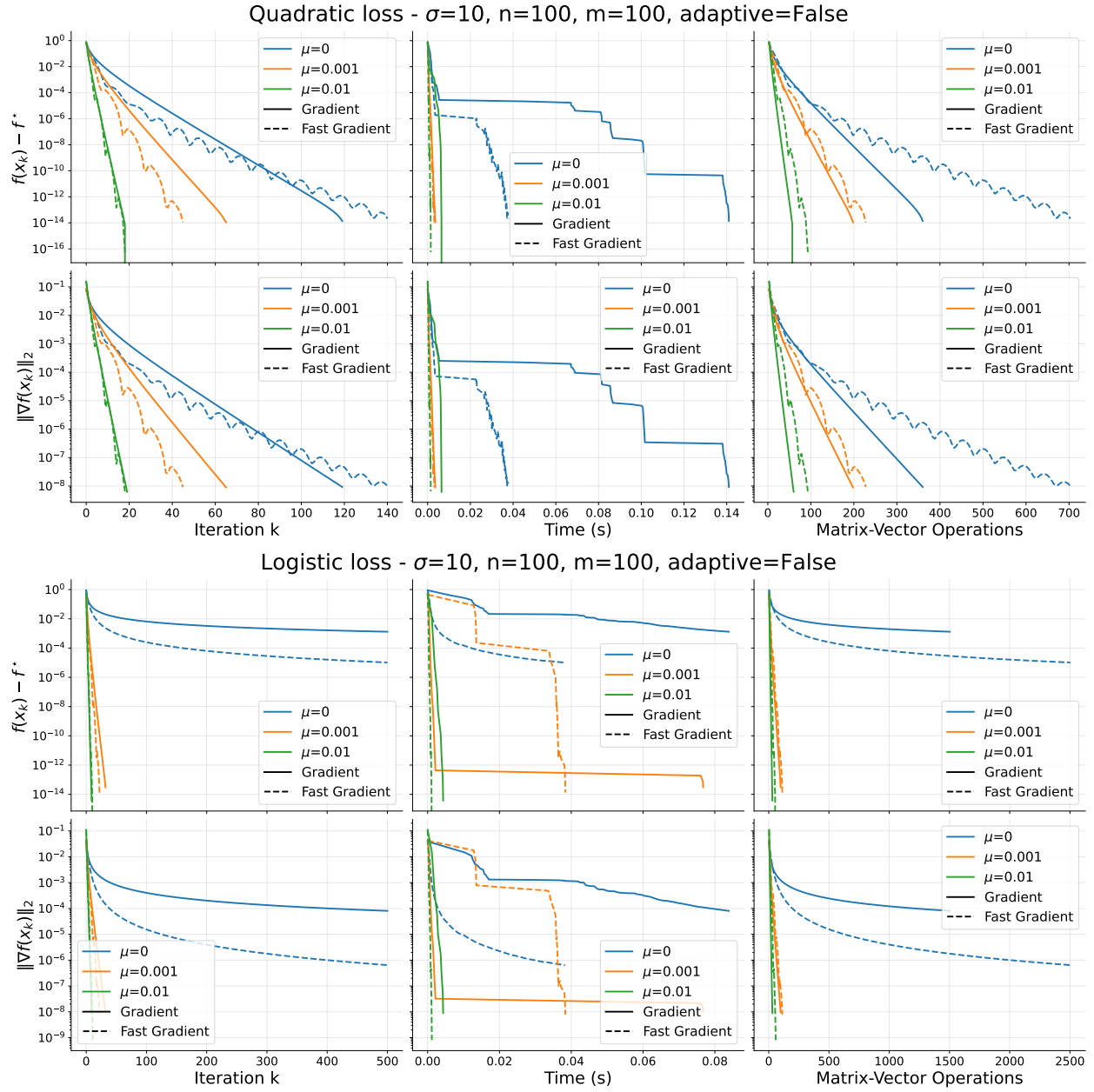


Figure 2: Smooth gradient methods. $n = 100, m = 100, \sigma = 10, \text{adaptive}=\text{False}$

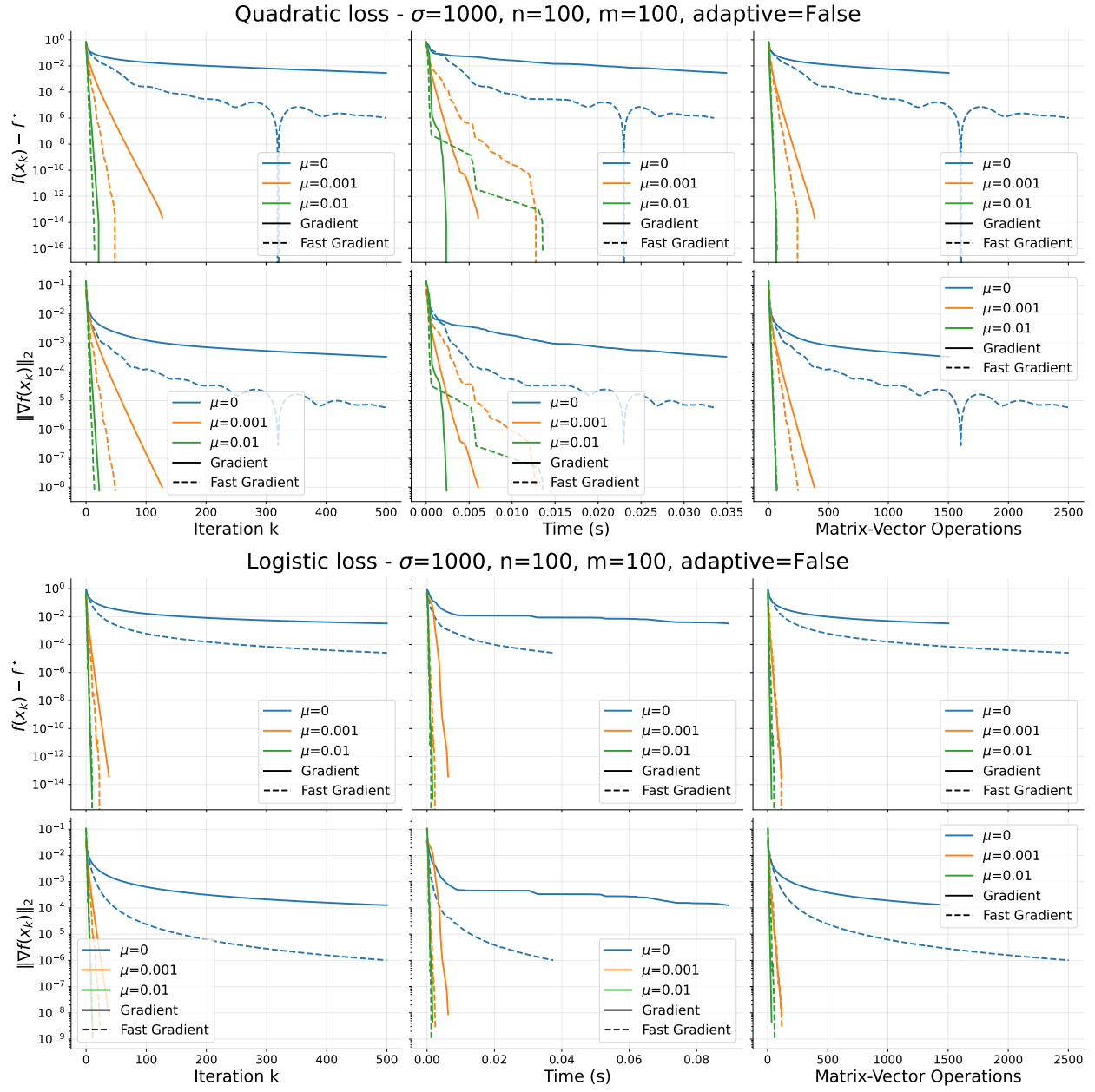


Figure 3: Smooth gradient methods. $n = 100, m = 100, \sigma = 1000, \text{adaptive}=\text{False}$

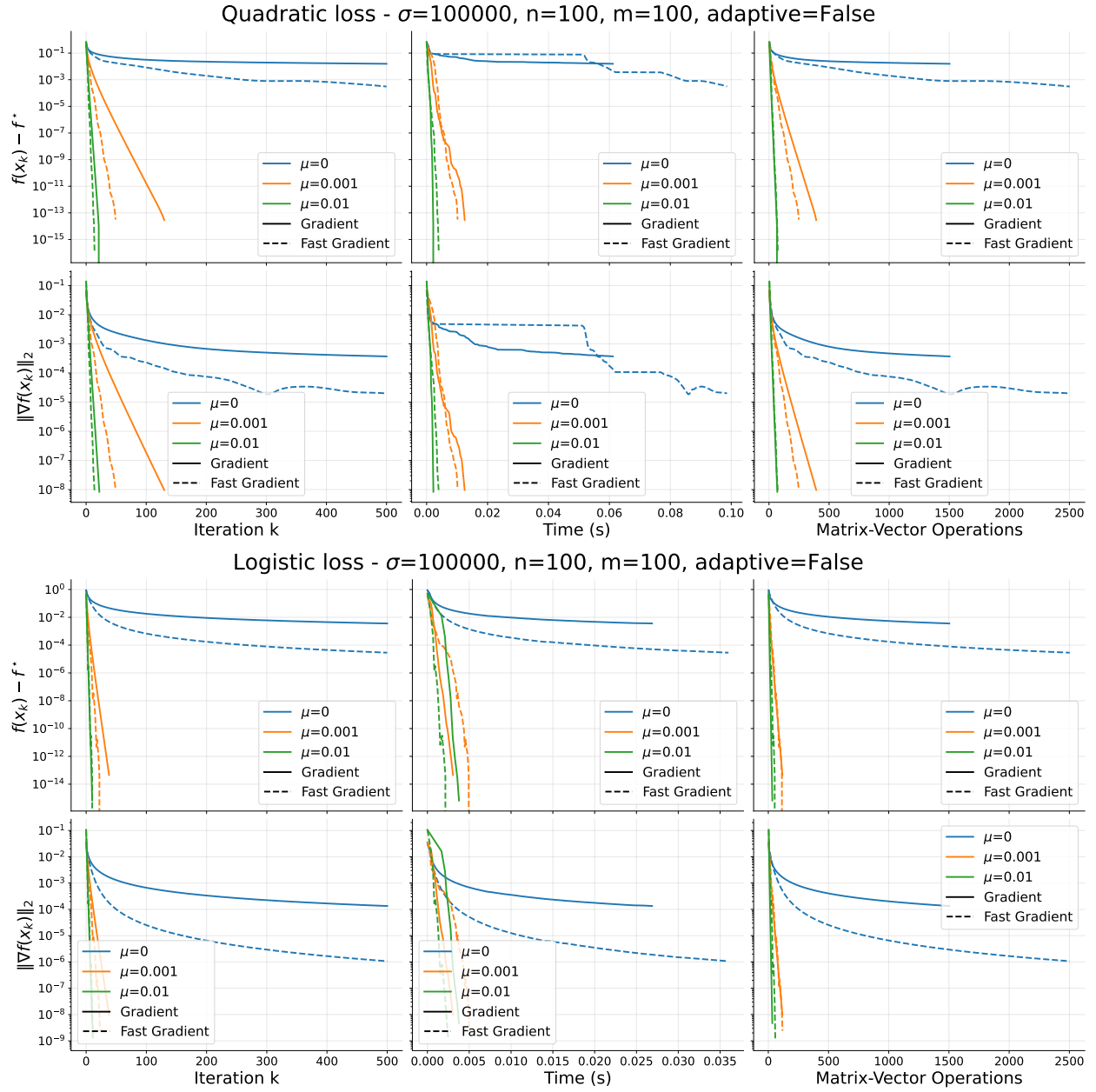


Figure 4: Smooth gradient methods. $n = 100, m = 100, \sigma = 100000, \text{adaptive}=\text{False}$

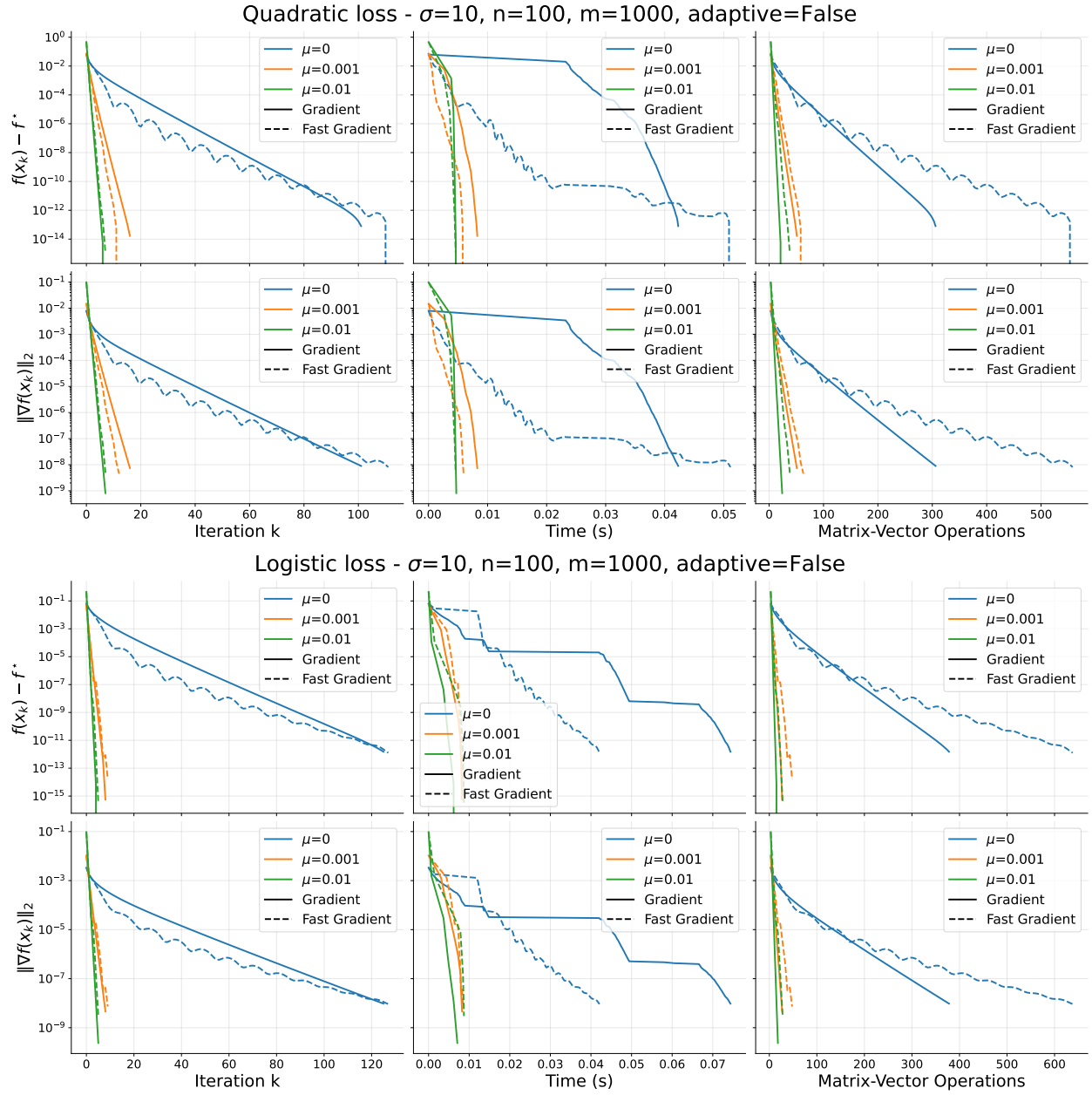


Figure 5: Smooth gradient methods. $n = 100$, $m = 1000$, $\sigma = 10$, $\text{adaptive}=\text{False}$

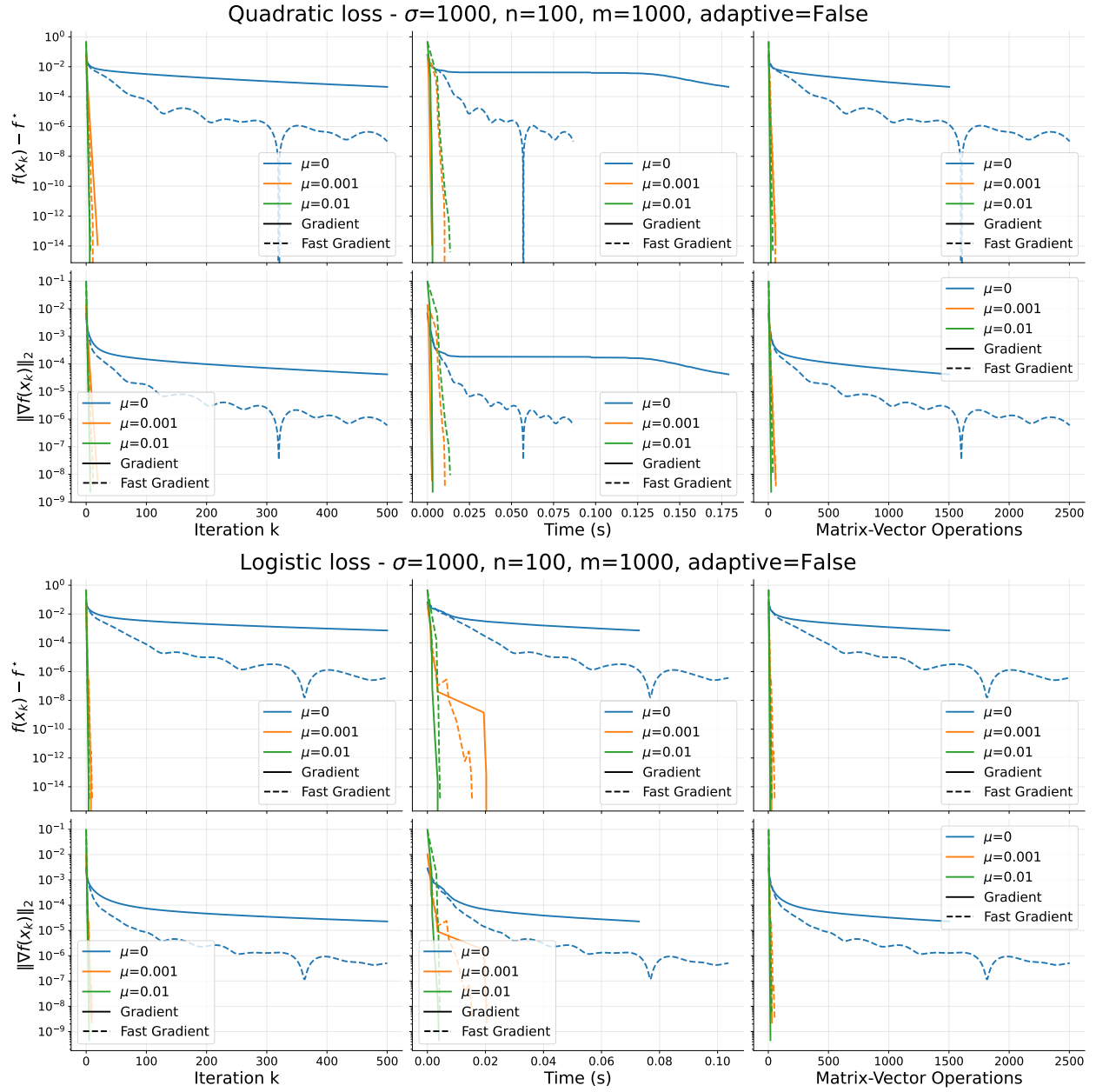


Figure 6: Smooth gradient methods. $n = 100, m = 1000, \sigma = 1000, \text{adaptive}=\text{False}$

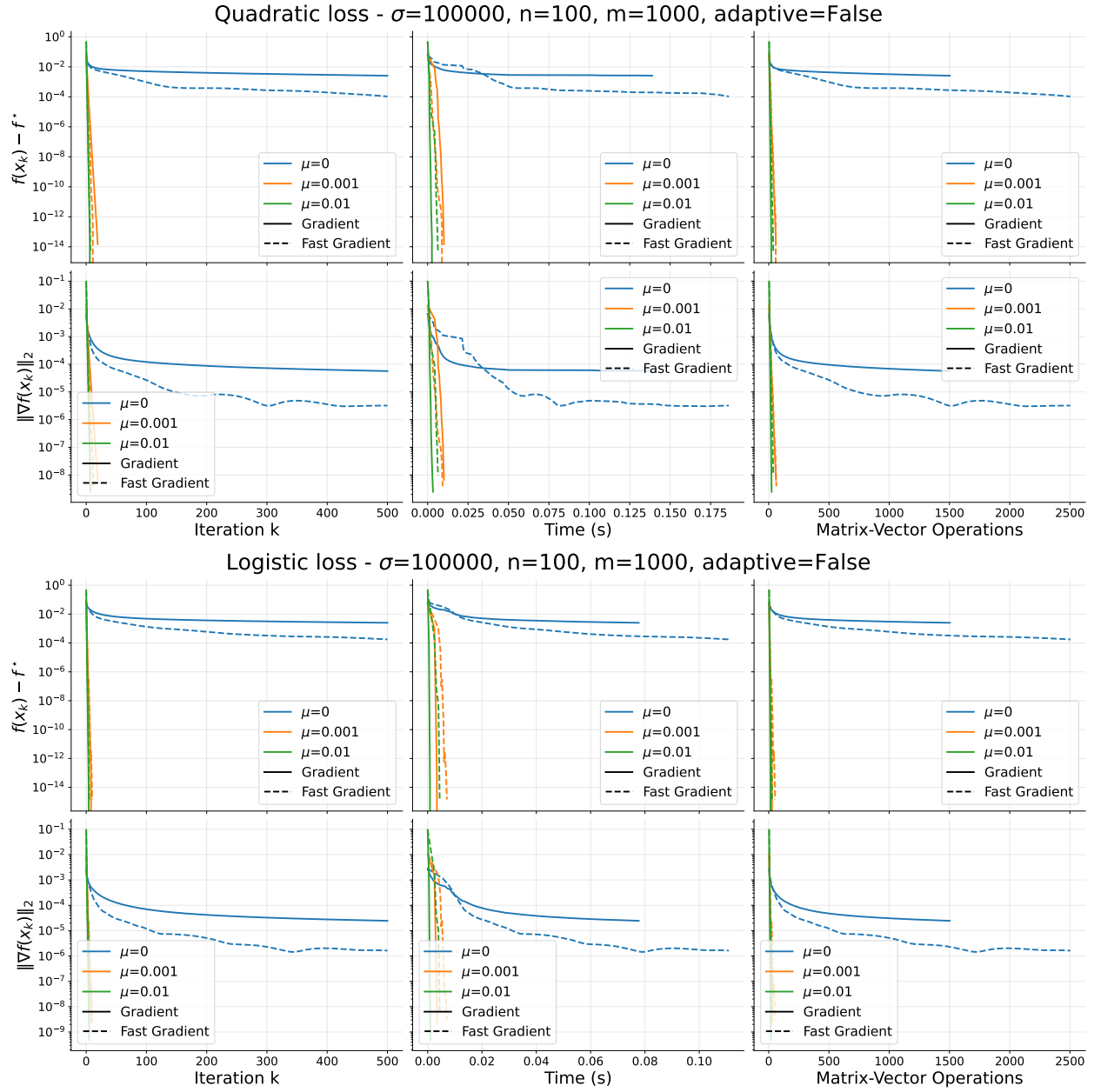


Figure 7: Smooth gradient methods. $n = 100, m = 1000, \sigma = 100000$, $\text{adaptive}=\text{False}$

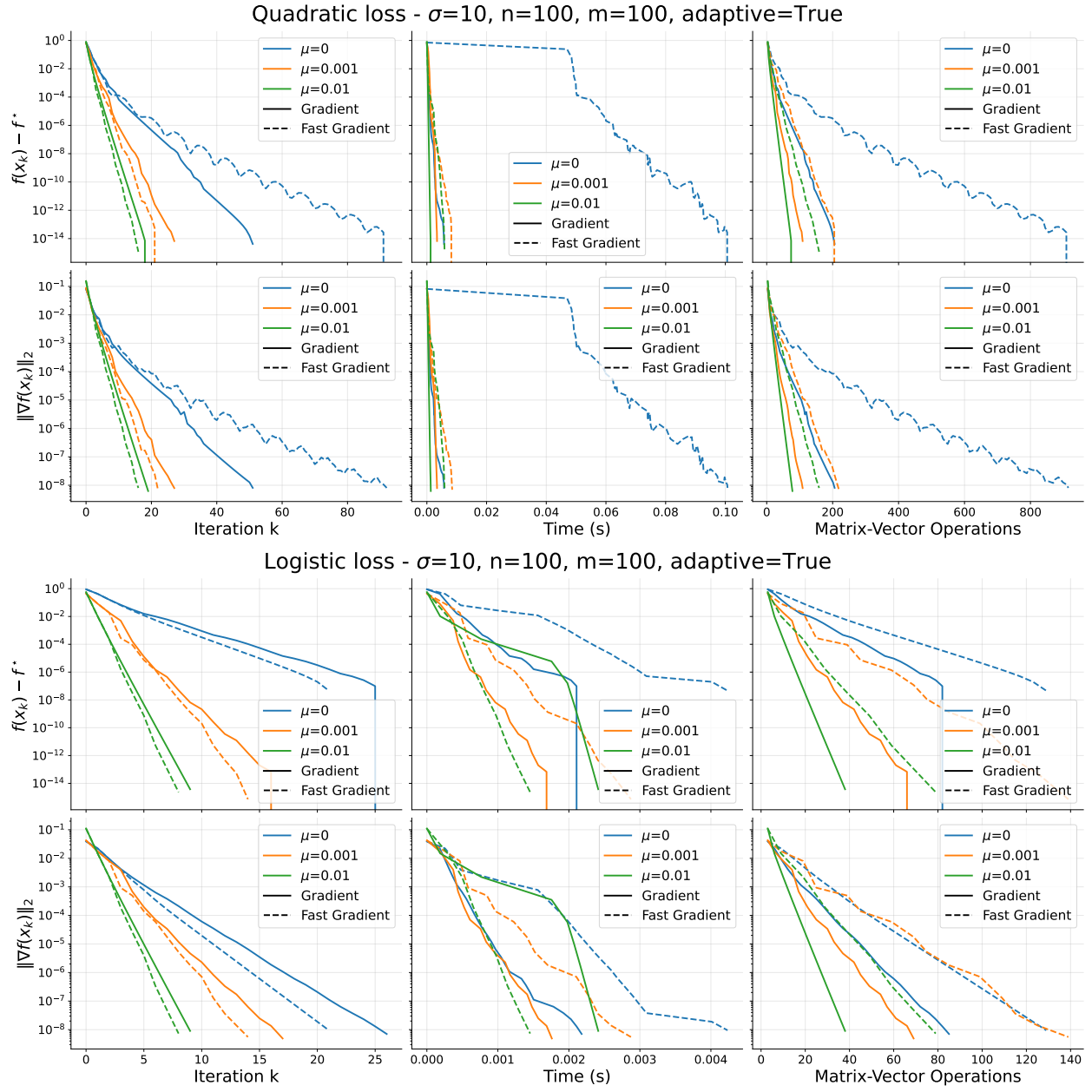


Figure 8: Smooth gradient methods. $n = 100, m = 100, \sigma = 10$, $\text{adaptive}=\text{True}$

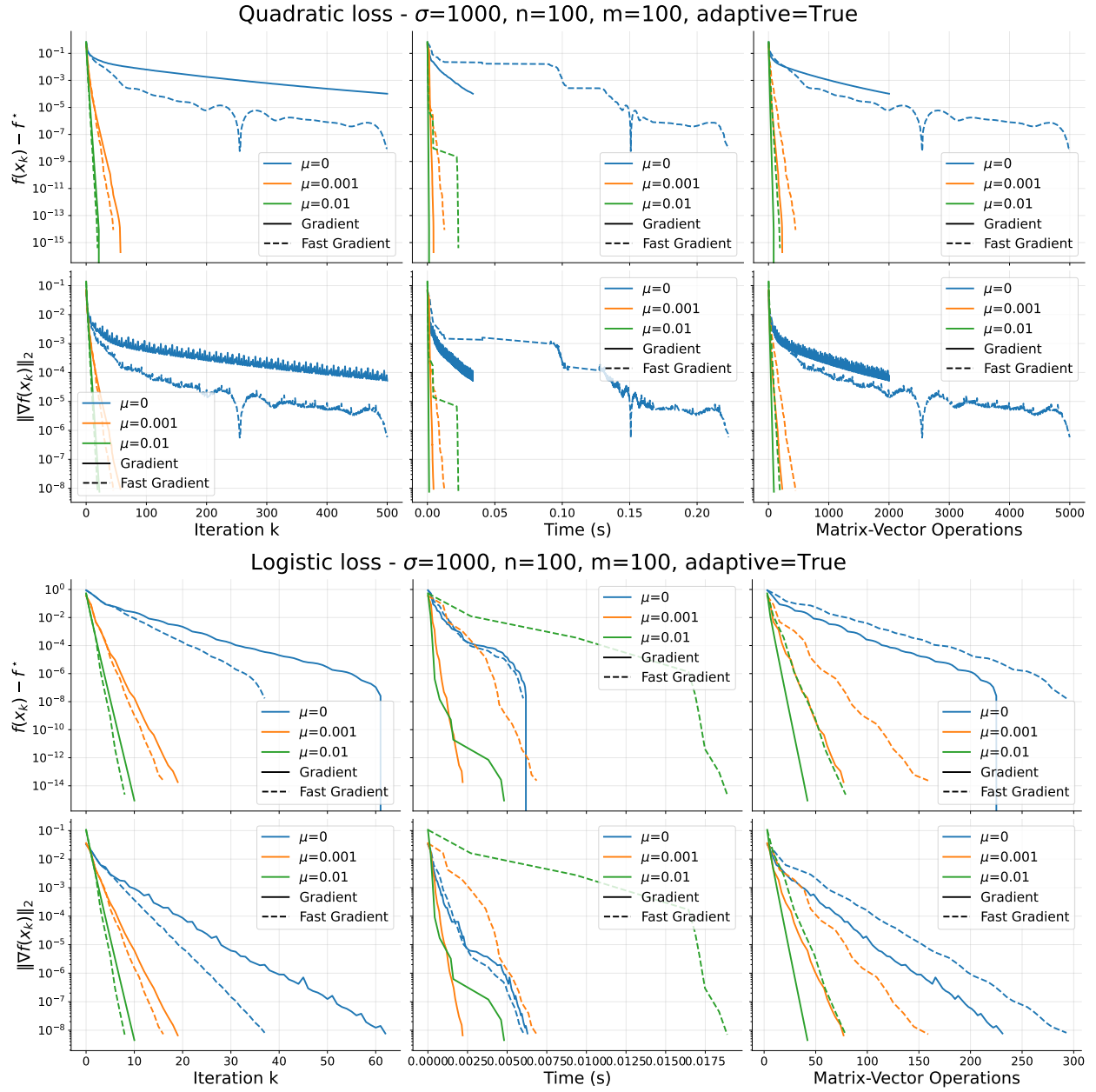


Figure 9: Smooth gradient methods. $n = 100$, $m = 100$, $\sigma = 1000$, $\text{adaptive}=\text{True}$

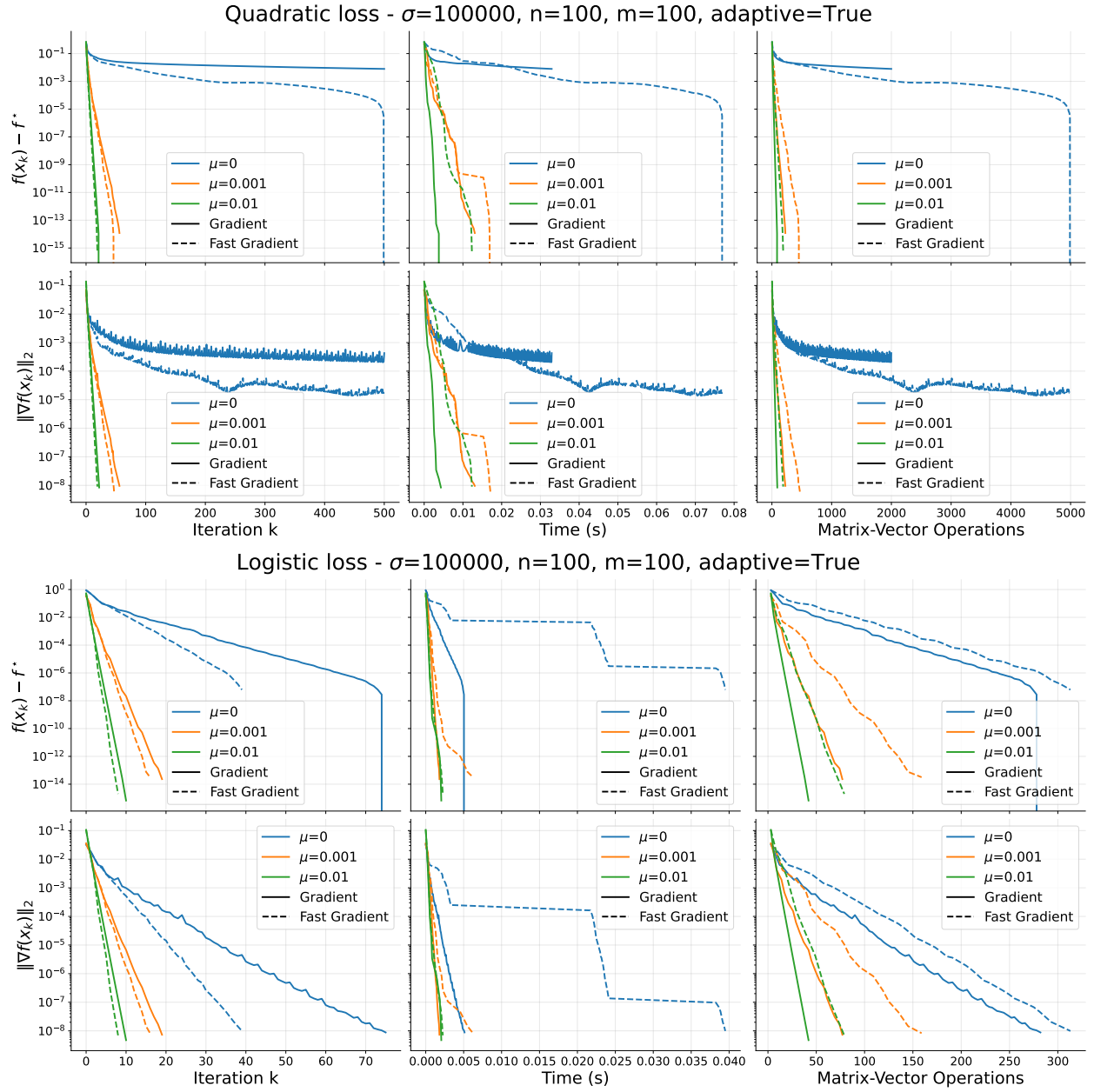


Figure 10: Smooth gradient methods. $n = 100, m = 100, \sigma = 100000$, $\text{adaptive}=\text{True}$

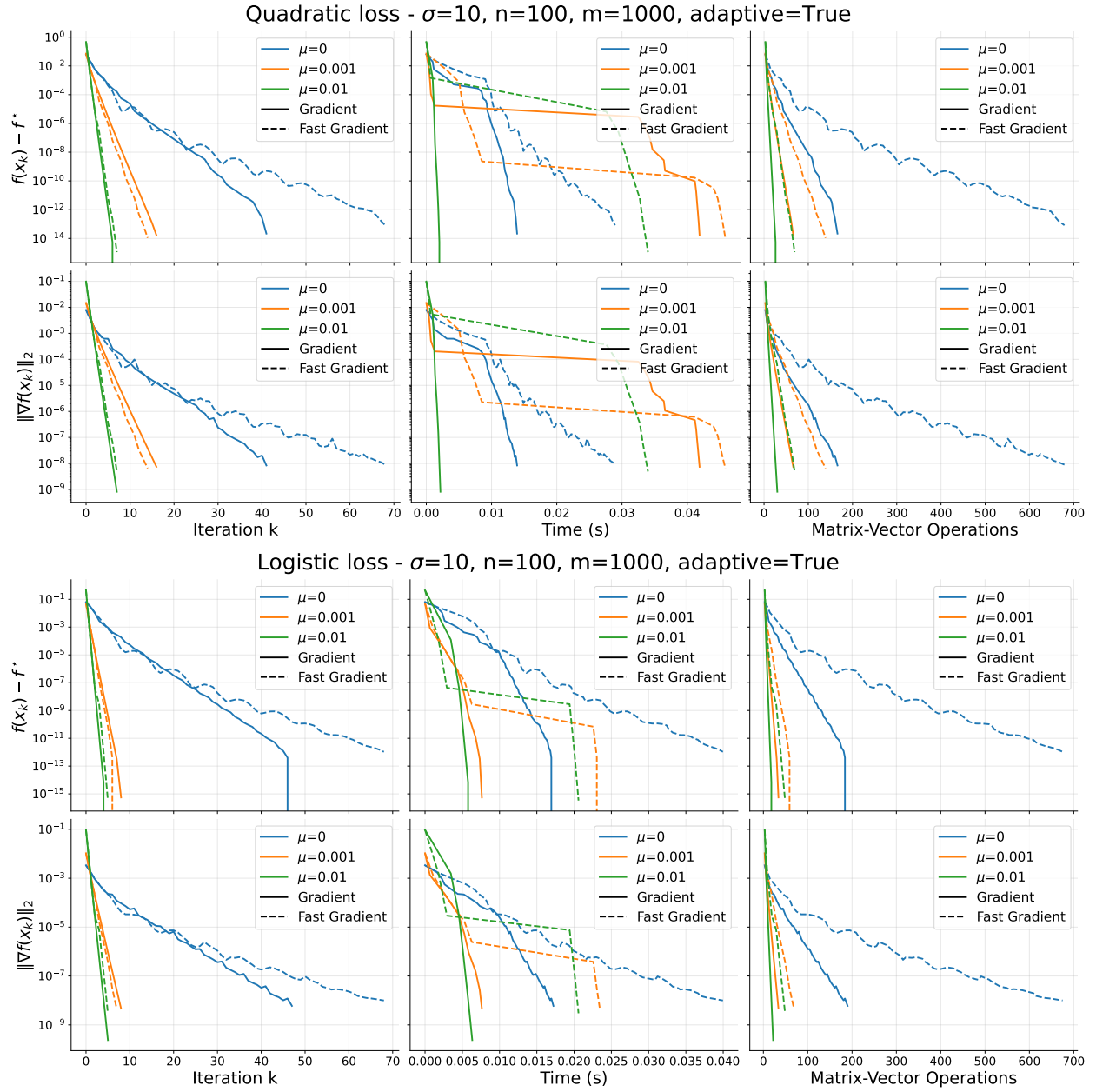


Figure 11: Smooth gradient methods. $n = 100, m = 1000, \sigma = 10, \text{adaptive}=\text{True}$

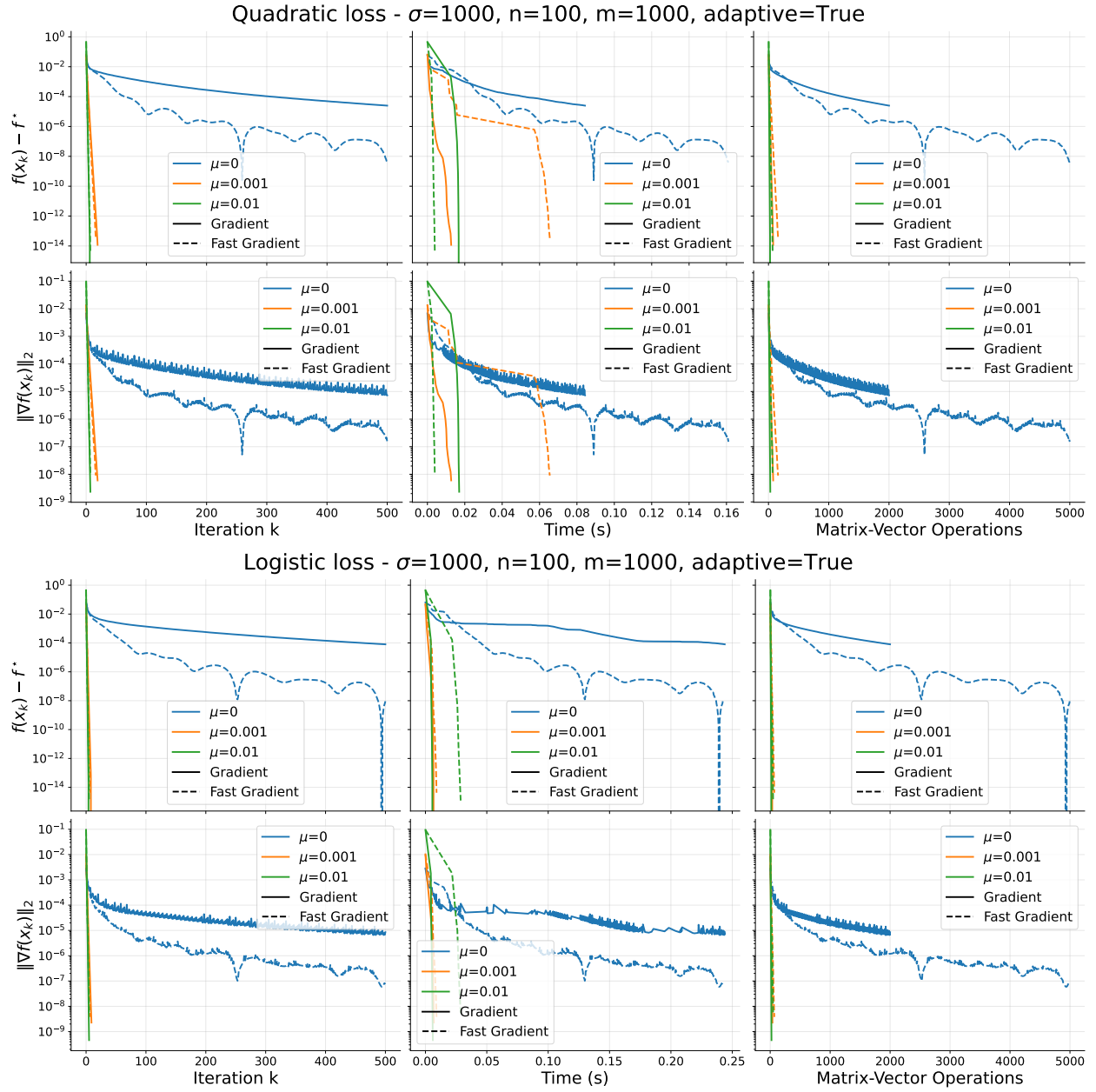


Figure 12: Smooth gradient methods. $n = 100, m = 1000, \sigma = 1000, \text{adaptive}=\text{True}$

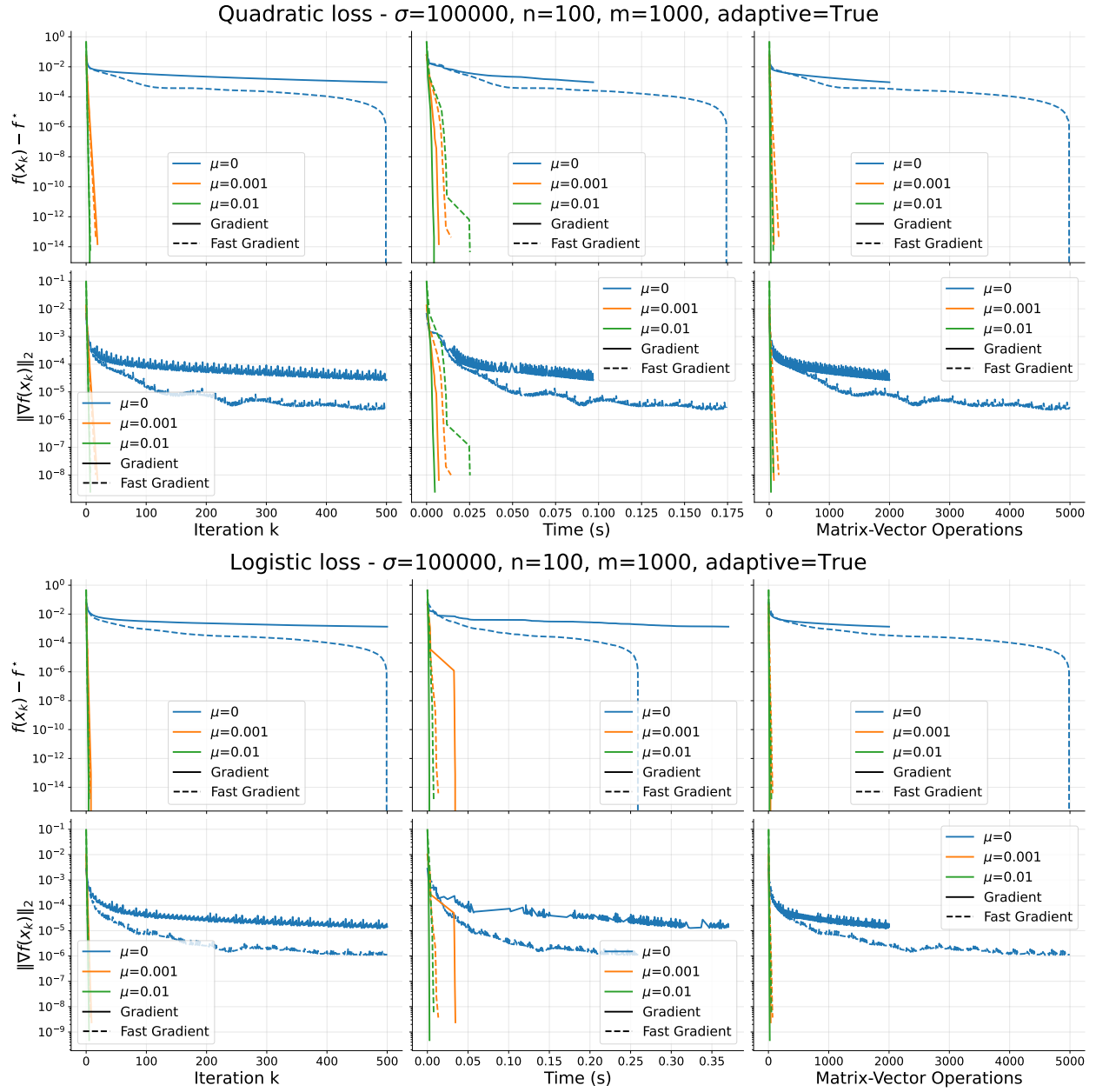


Figure 13: Smooth gradient methods. $n = 100, m = 1000, \sigma = 100000, \text{adaptive}=\text{True}$

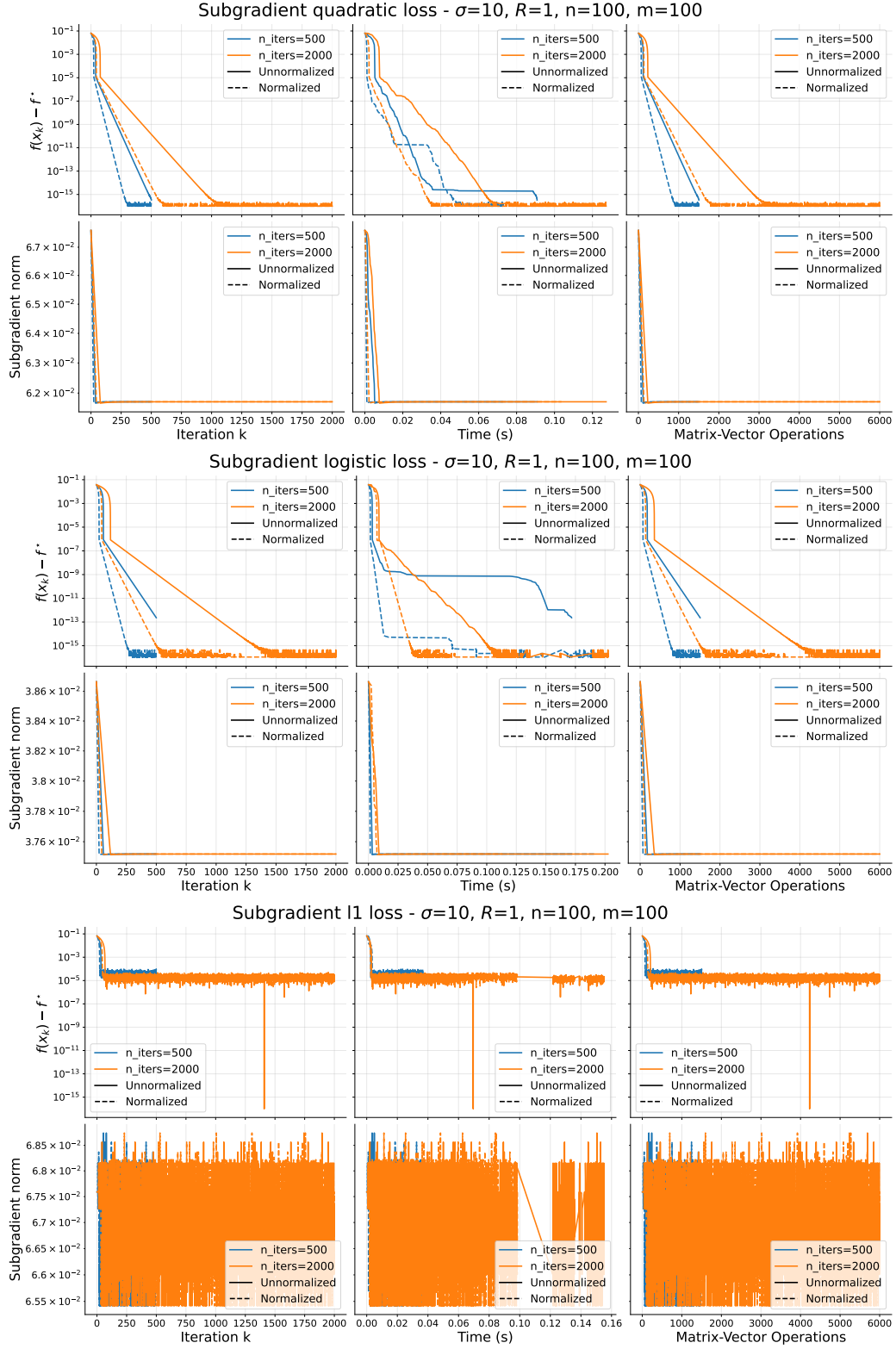


Figure 14: Subgradient methods. $n = 100, m = 100, \sigma = 10, R = 1$.

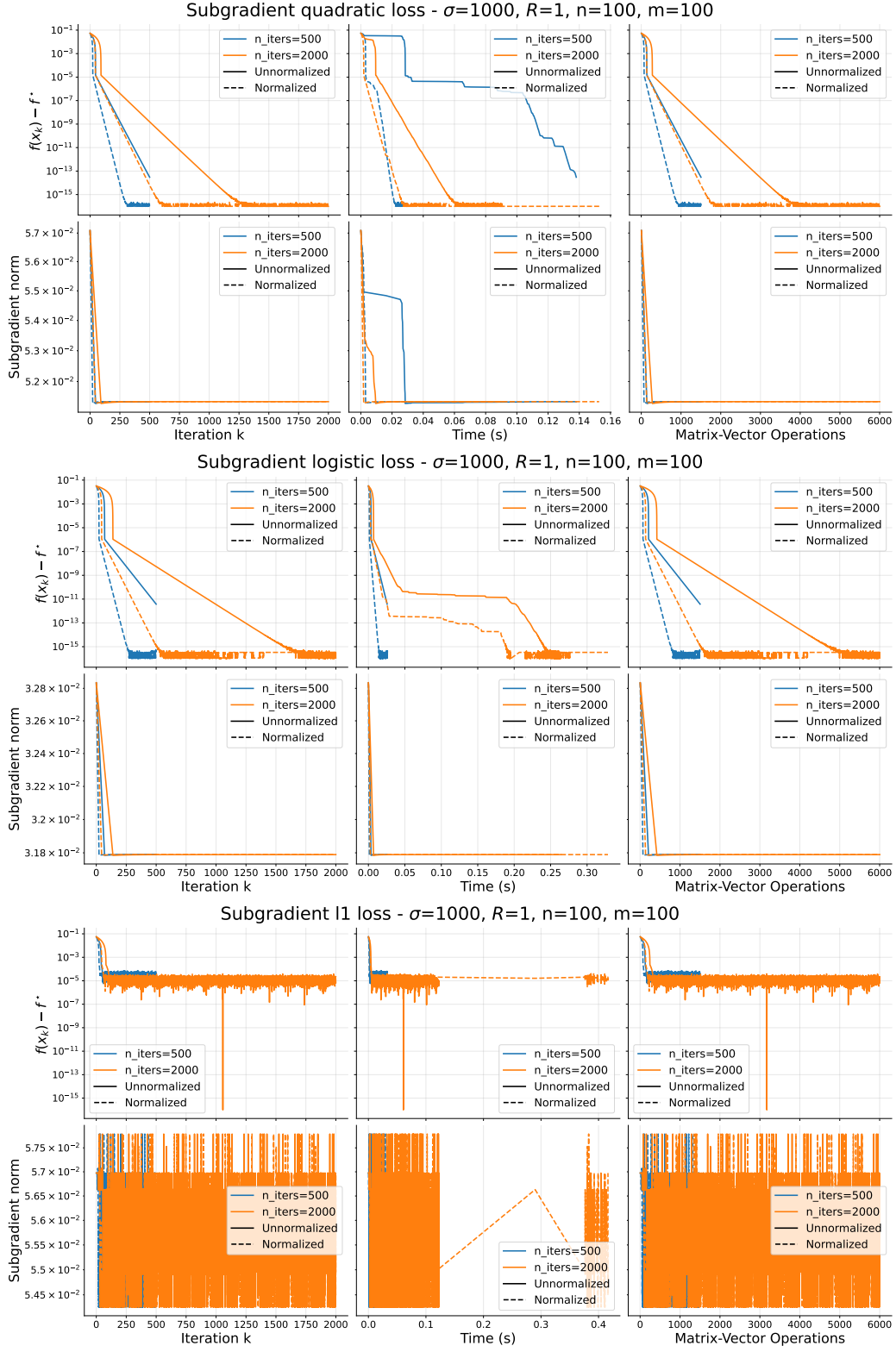


Figure 15: Subgradient methods. $n = 100, m = 100, \sigma = 1000, R = 1$.

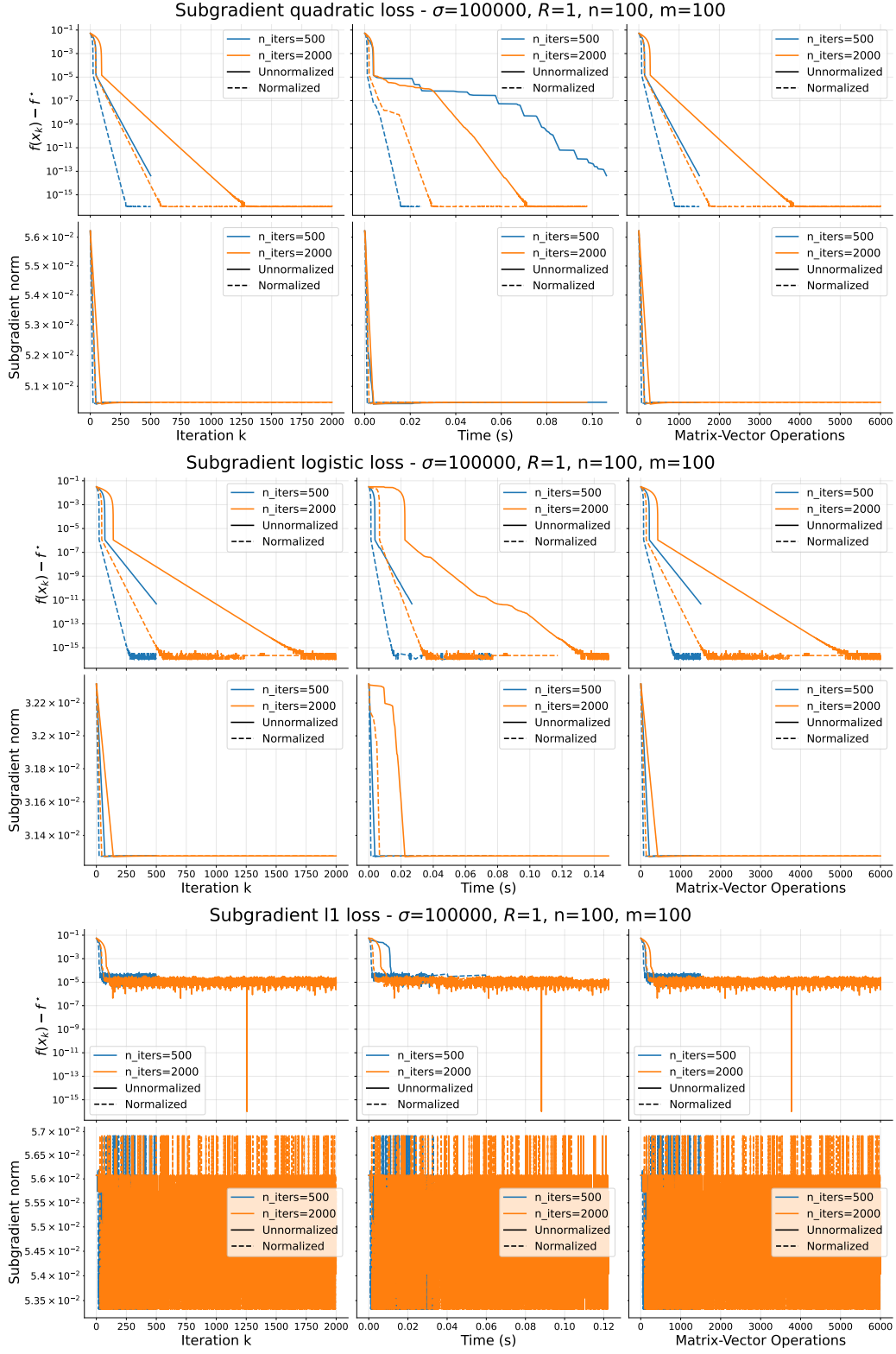


Figure 16: Subgradient methods. $n = 100, m = 100, \sigma = 100000, R = 1$.

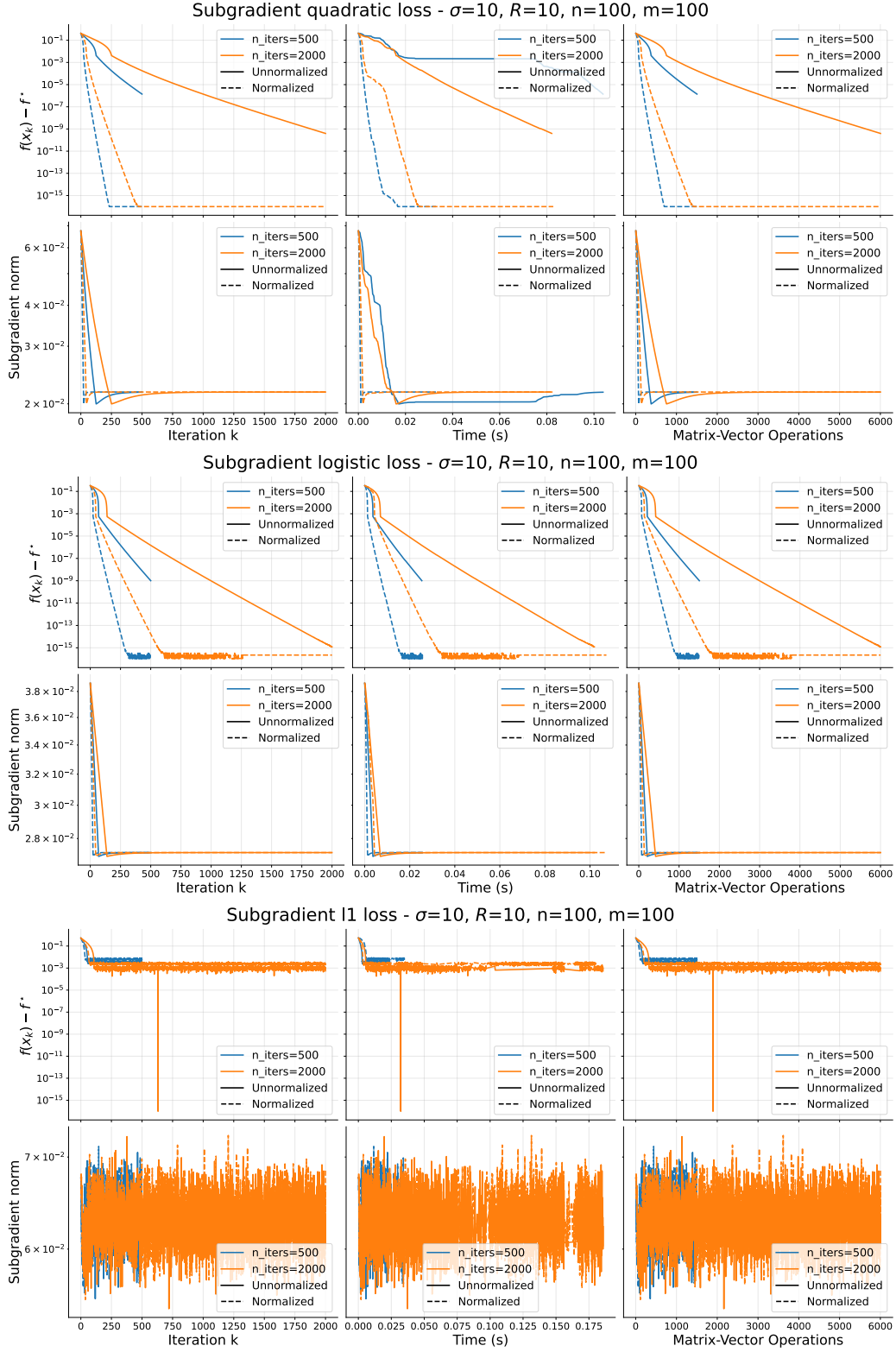


Figure 17: Subgradient methods. $n = 100, m = 100, \sigma = 10, R = 10$.

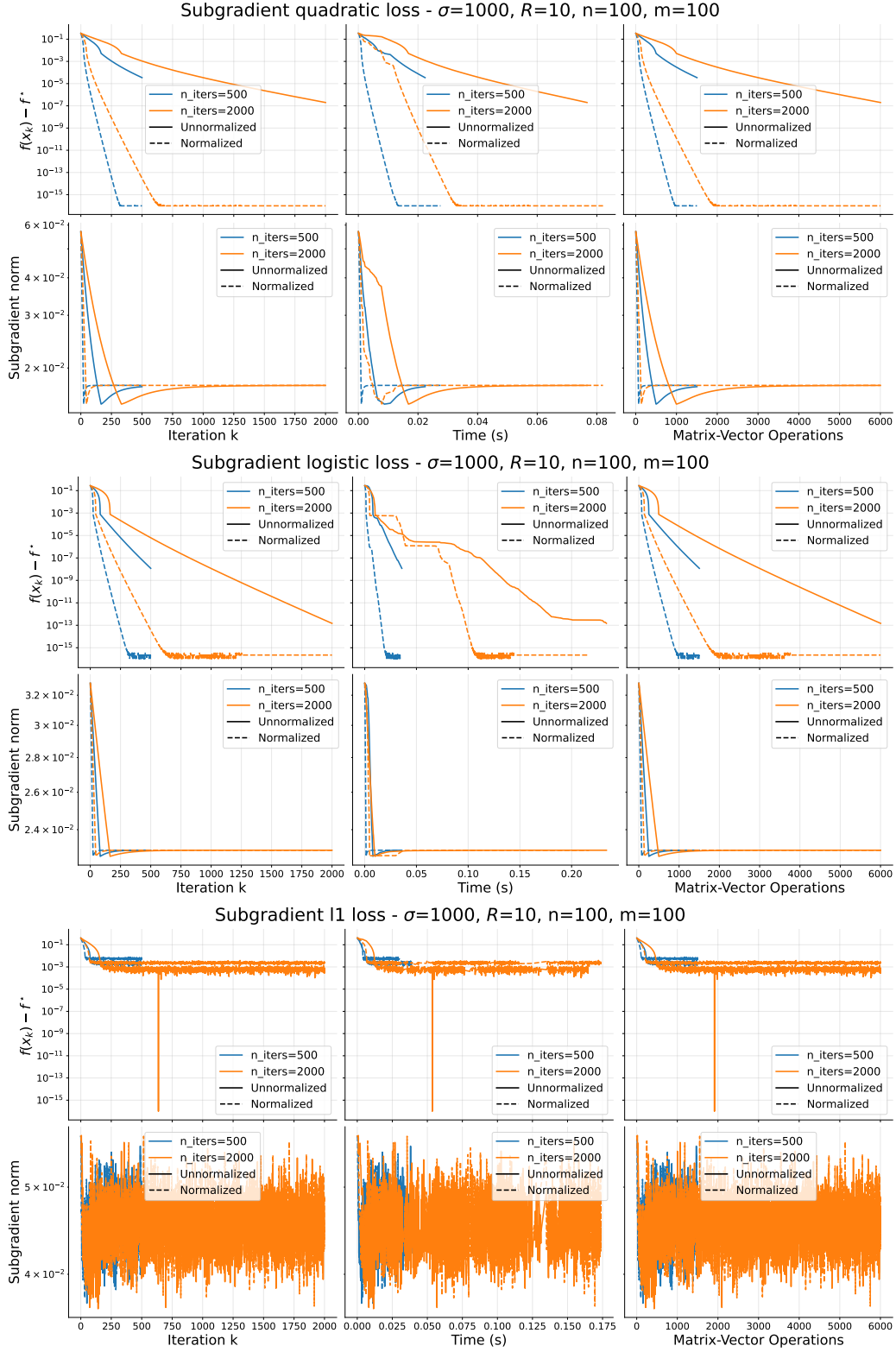


Figure 18: Subgradient methods. $n = 100, m = 100, \sigma = 1000, R = 10$.

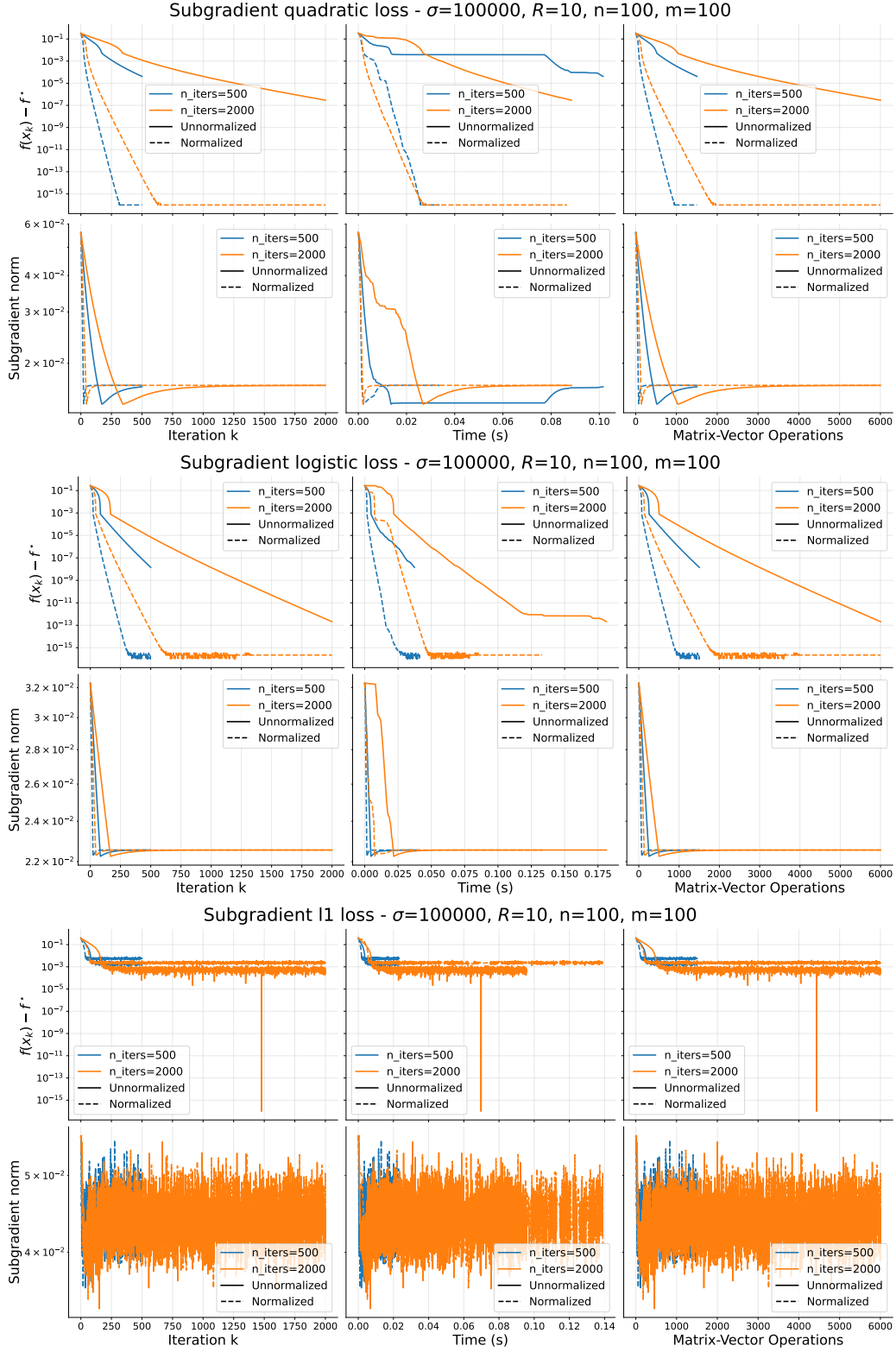


Figure 19: Subgradient methods. $n = 100, m = 100, \sigma = 100000, R = 10$.

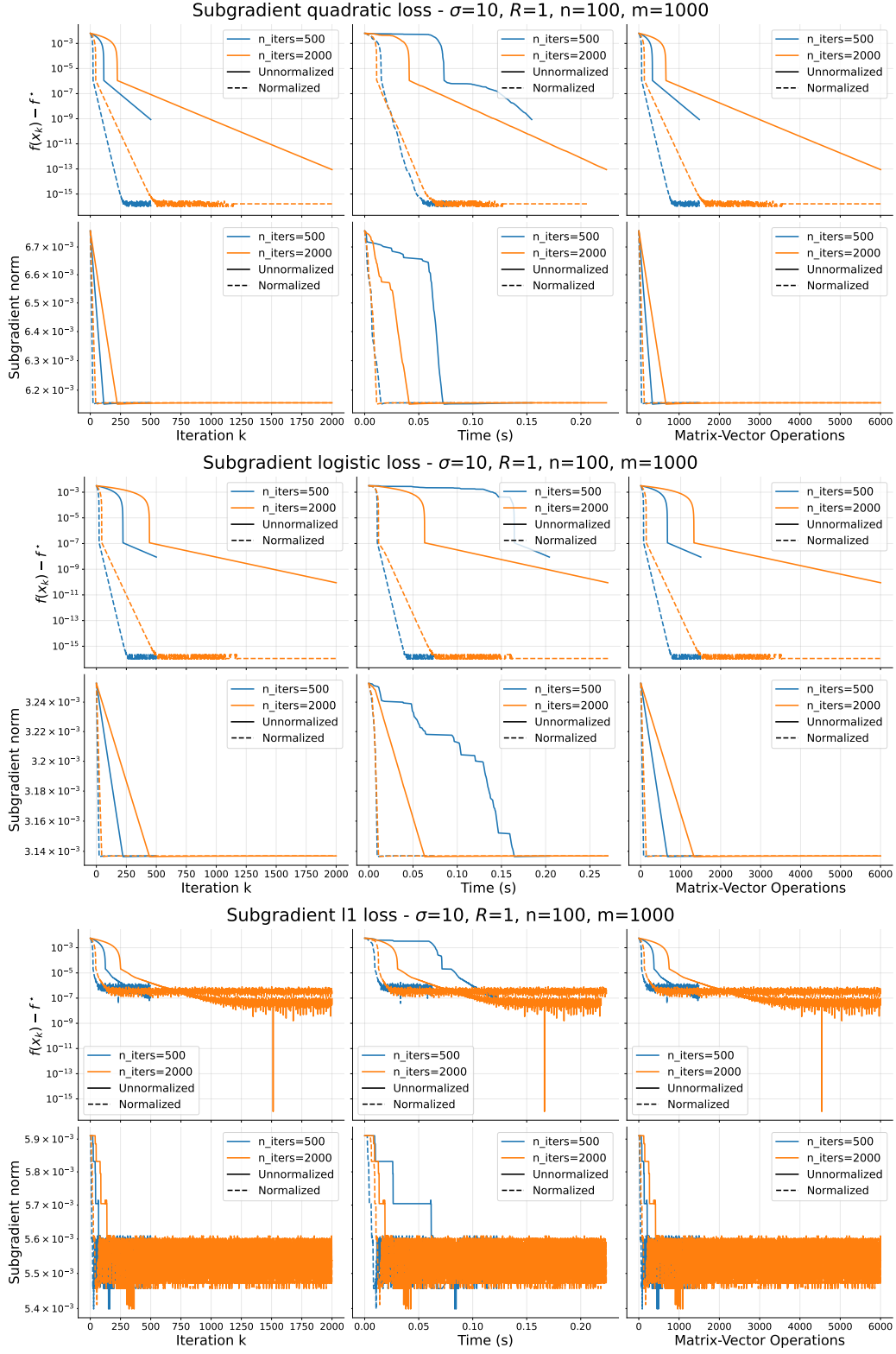


Figure 20: Subgradient methods. $n = 100, m = 1000, \sigma = 10, R = 1$.

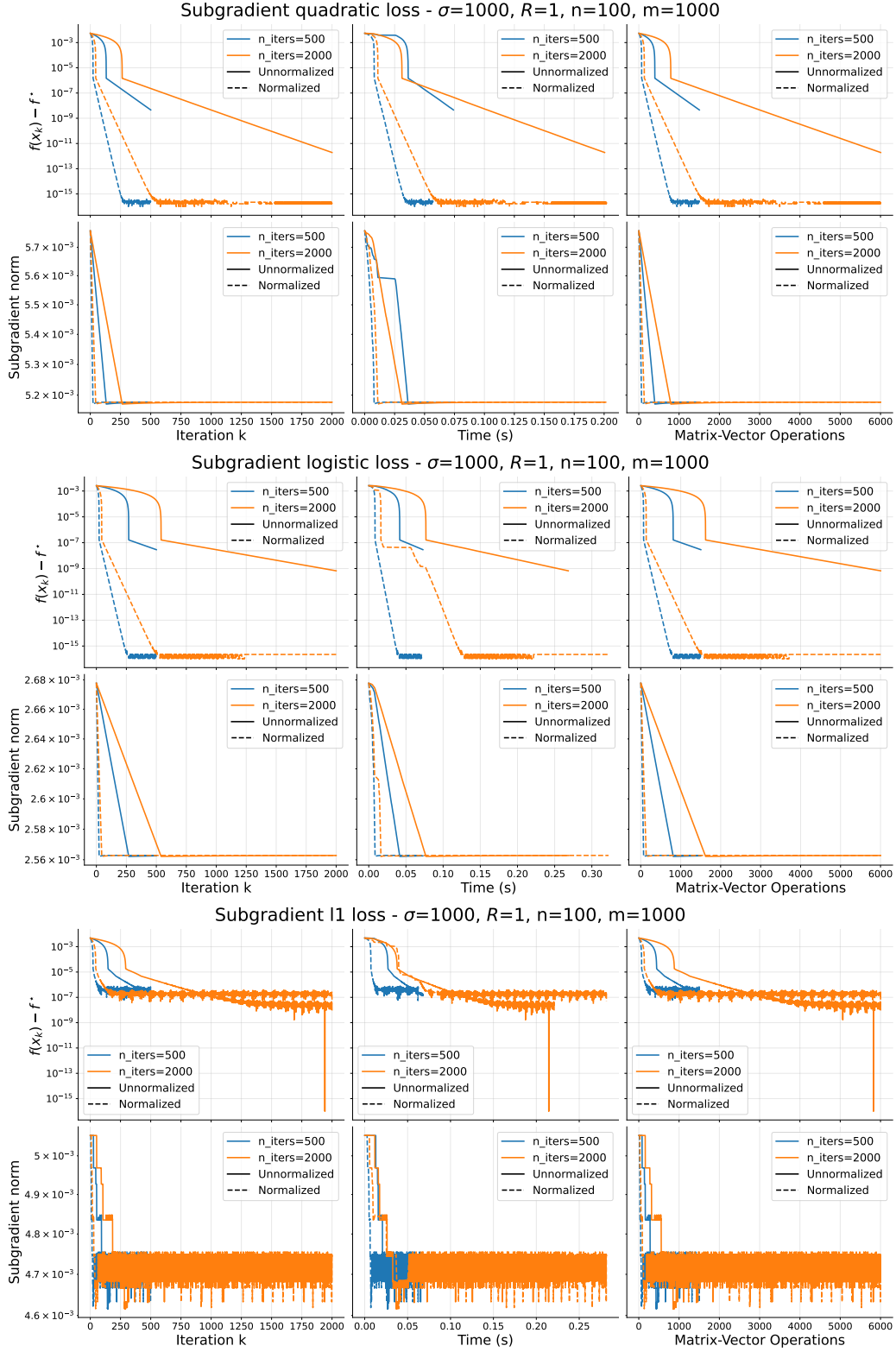


Figure 21: Subgradient methods. $n = 100, m = 1000, \sigma = 1000, R = 1$.

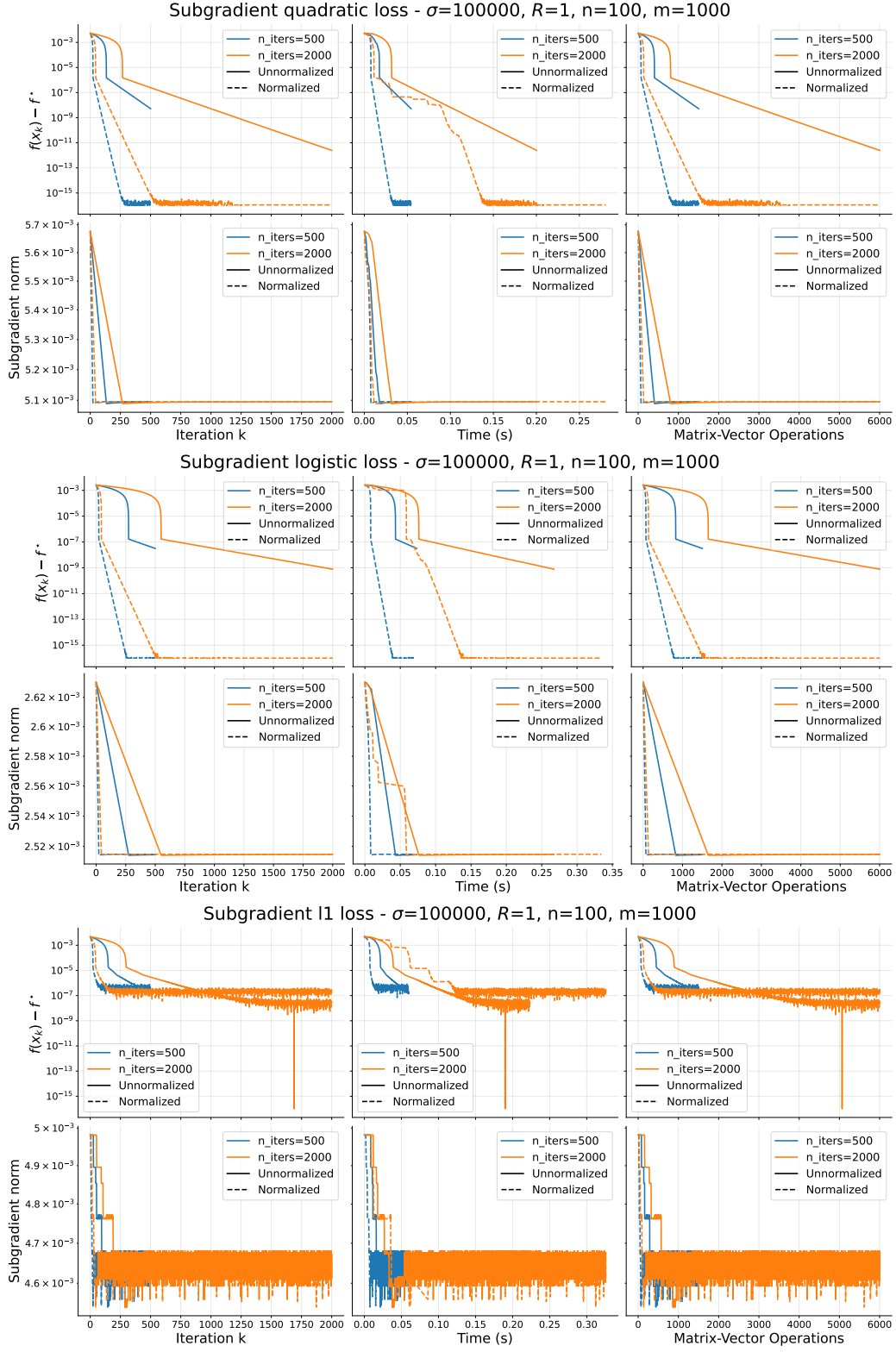


Figure 22: Subgradient methods. $n = 100, m = 1000, \sigma = 100000, R = 1$.

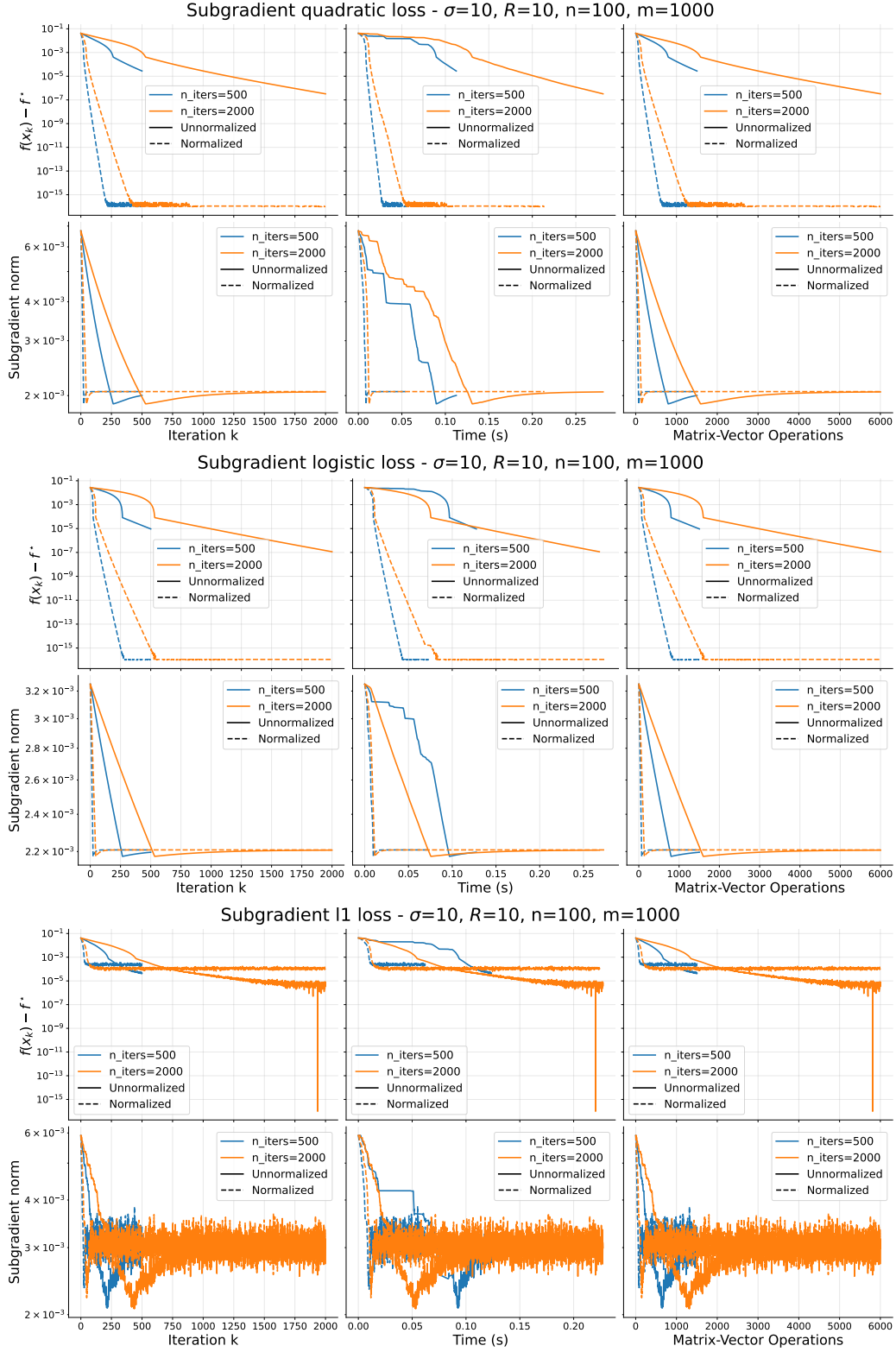


Figure 23: Subgradient methods. $n = 100, m = 1000, \sigma = 10, R = 10$.

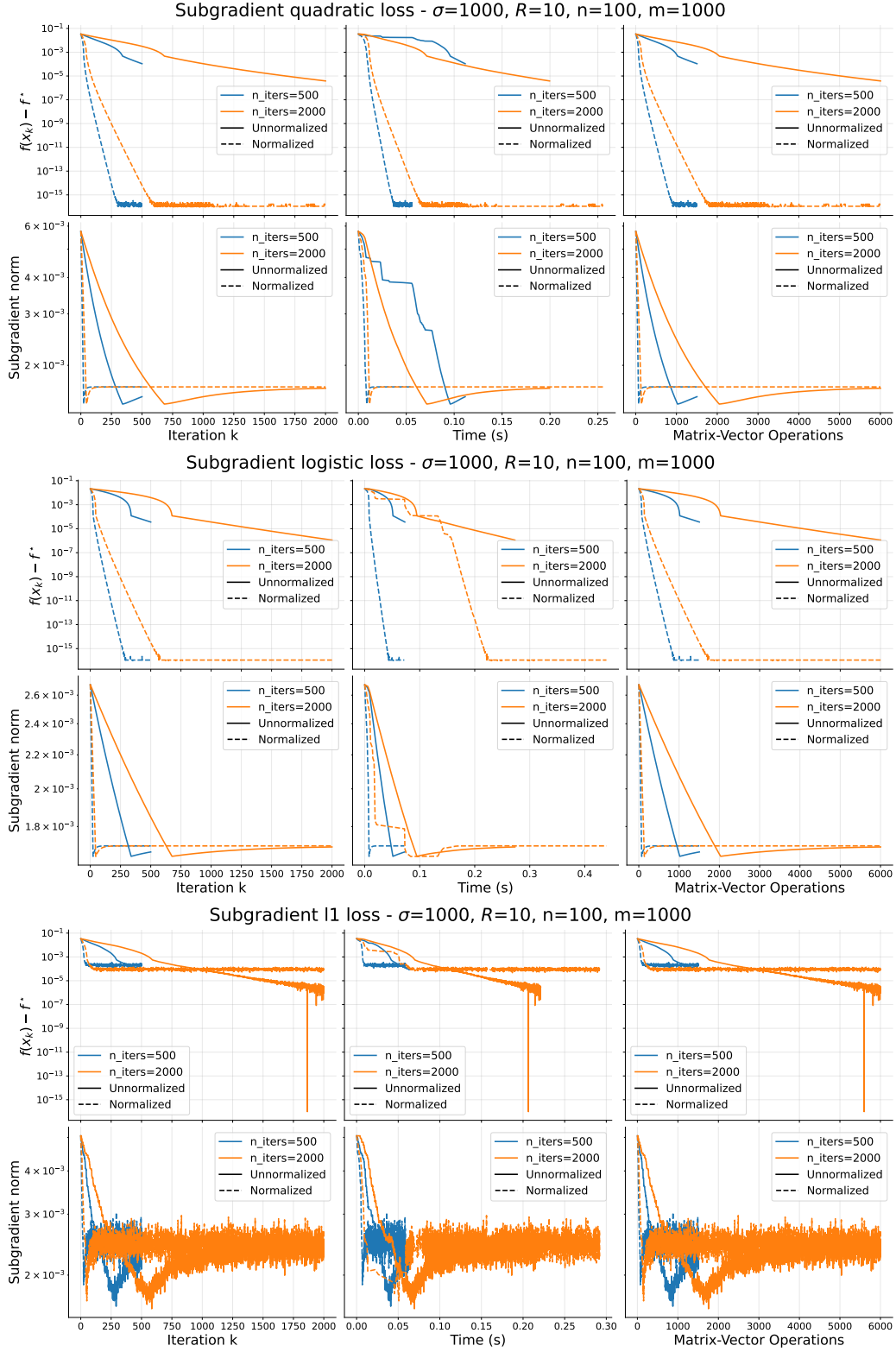


Figure 24: Subgradient methods. $n = 100, m = 1000, \sigma = 1000, R = 10$.

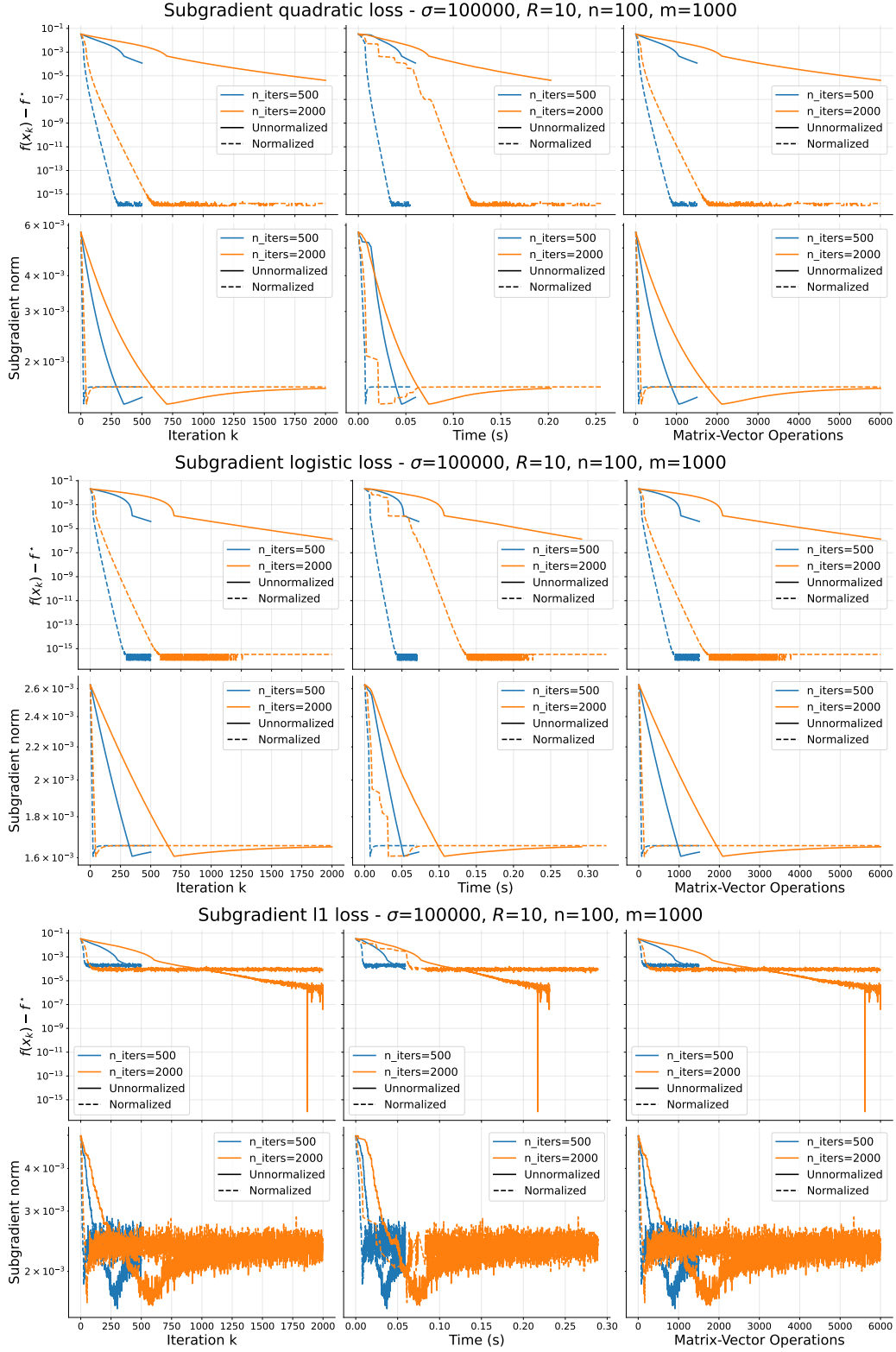


Figure 25: Subgradient methods. $n = 100$, $m = 1000$, $\sigma = 100000$, $R = 10$.